

Microcontroller Hackathon Challenge - 2026

Overview

The main challenge is to design a microcontroller for the edge with compiled software, with a focus on validation, testing and a well engineered design.

This is a full-stack embedded design challenge to complete over a couple of days. It is an ambitious undertaking with many valid paths, so there is no single correct solution; builds may fail and errors will be raised, but debugging is a large part of engineering on a highly technical project. We are looking for sound engineering judgement and design.

THEME

Performance at the edge with a hardware and software focus.

How you interpret this theme is up to you. It might mean the smallest possible core, the fastest core, a design tuned for streaming data throughput, low power operation, a specialised instruction set built around a particular workload, hardware peripherals, or multi core. Choose an angle that interests you and let it guide your trade-offs.

Expect questions like: How small can the design be and how does that contrast with performance? How do you prove it works? What trade-offs did you make, and why? Pick a path, explore it, and be able to walk through your reasoning. A half-finished or smaller build with a clear rationale behind it counts for a lot.

Goals

These goals build on each other. You are not expected to finish all of them. Depth and rigour matter more than raw completion.

1. **Build a CPU:** implement a working core for your chosen ISA.
2. **Compile a custom program:** get real software running on your core.
3. **Design your tests:** prove correctness with a deliberate test strategy.
4. **Design your benchmarks:** measure performance and area objectively.
5. **Design your software:** write meaningful workloads for the core.
6. **Optimise your design:** improve area, speed, or power with evidence.

***Note:** All tools are permitted. The one rule is that you must be able to defend your work. If you cannot explain a design decision or a line of generated code, it does not count in your favour during the mini audit.*

Deliverables

To keep judging fair within the time available, the challenge is assessed on:

- **A short 2-minute mock audit** near the end of the final day. Designs need not be complete at this stage; this focuses on your understanding, your plan and process, and how you approached testing and design.
- **A short 5 to 10 minute video** presenting the project.
- **Optionally, a link to a git repository.**

This is deliberately light so you can spend your time building rather than on paperwork.

***Note:** Submission links for your video and repository will be shared with you during the event.*

Mini audit

For high-reliability domains such as automotive and medical, a certification audit is common industry practice. This mock audit mirrors that: judges will ask you to walk through what you built, how you tested it, and why you trust it. Come prepared to show evidence, not just claims.

Final video

This is the main way to show your project and defend its technical execution. Keep it focused on the design, the testing approach, and the results. It does not need to be a sales pitch; the format is engineer to engineer.

Optional git links

Optional but recommended. Judges will only check out your code or scripts if something in the video needs clarification.

Judging

Scoring uses round numbers to stay transparent. Each category is scored against the rubric below (total **70 points**). We reward clear evidence of good engineering and honest reasoning over polish, and hold every team to the same

standard. The bands describe a spectrum, and most teams will land in the middle.

Category	Strong	Solid	Partial	Focused elsewhere
Mini audit	20–15 Confident, evidence-backed answers; defends every part.	14–10 Understands most of the design; some gaps or hand-waving.	9–5 High-level description; struggles to justify decisions.	4–0 Cannot explain or defend the work.
Technical quality	20–15 Well-architected core running compiled software; clean, readable implementation with sound, justified design trade-offs.	14–10 Working or nearly-working core with a reasonable structure; some rough edges or understood correctness gaps.	9–5 Simulates in parts; design is basic or unrefined and runs no real compiled software.	4–0 Little working hardware; minimal evidence of design thinking.
Process quality	10–8 Deliberate, documented test and validation strategy with real coverage.	7–5 Some testing present but ad hoc or incomplete.	4–2 Basic sanity checks only, undocumented.	1–0 Little or no evidence of testing.
Presentation quality	10–8 Clear, well-structured video and audit; easy to follow the technical details.	7–5 Understandable but disorganised or unclear in places.	4–2 Present but hard to follow or missing key parts.	1–0 No presentation delivered.
Novelty	10–8 Original ISA, optimisation, or approach with clear insight.	7–5 Some original thinking on a conventional base.	4–2 Minor tweaks to an existing design.	1–0 Purely derivative.

Getting started

You do not need any prior CPU-design, compiler, or FPGA experience; the tutorials teach all of that. A working familiarity with Verilog, C, and the command line is all you need to get going.

1. **Follow the tutorial stages** (Setup → Data Operations → Control Flow → Function Calls). Each stage is a full vertical slice, from ISA to a running C program, so you always have something that works.
2. **Reach for the stretch goals** only once the basics are solid. They are bonus, not baseline.
3. **Iterate quickly.** Keep the changes small and test as you go. Work with your team and *divide and conquer* where possible.

Get *something* end-to-end working early, then improve it. A tiny CPU that runs a real program beats an ambitious design that never builds.

Support during the event

Mentors and organisers are around throughout. Ask questions early and often. If you hit a wall with tooling, a teammate, or a design decision, flag it and we will help you find a way forward.

Tips for a strong submission

- **Walk through your process.** Explain what you built, what you tried, and what you learned, including the dead ends.
- **Show your evidence.** A passing simulation, a waveform, or a short test log speaks louder than a claim.
- **Be honest about gaps.** “We know X doesn’t work yet, and here’s why” scores better than pretending it does.
- **Keep your earlier cores.** You don’t need a single design; keeping a simple baseline alongside a more ambitious variant lets you compare area, speed, and trade-offs, which is strong evidence of engineering judgement.
- **Keep it simple.** Your video does not need to be slick; clear and direct is enough.

A final word

Whatever you build, you will leave knowing more about CPUs, compilers, and testing than when you arrived. Look after your teammates, take breaks, and we look forward to seeing what you create.